

Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM
Ciência da Computação

MiniCCO

Núcleo Geométrico e Estrutural para Crescimento Arterial Simplificado

Ana Paula dos Santos Tadim, Heitor Augusto de Oliveira Ramos, Matheus Maia
Amorim

Professor: Rafael Alves Bonfim de Queiroz

Ouro Preto
6 de junho de 2026

Sumário

1	Introdução	1
1.1	Considerações iniciais	1
1.2	Instruções de compilação e execução	1
2	Descrição da implementação	2
2.1	Organização dos Arquivos	2
3	Exemplos de execução	4
4	Análise de Resultados	10
4.1	Análise do Relatório Final e a Equação de Nós	10
4.2	Cenário 1: Simulação com 1 Terminal	11
4.3	Cenário 2: Simulação com 5 Terminais	11
4.4	Cenário 3: Simulação com 10 Terminais	12
5	Discussão sobre Complexidade	12
5.1	Busca de Candidatos	12
5.2	Teste de Interseção	13
6	Considerações Finais	14

Lista de Figuras

1	Árvore arterial com 1 terminal ($N_{term} = 1$).	4
2	Árvore arterial com 2 terminais ($N_{term} = 2$).	5
3	Árvore arterial com 5 terminais ($N_{term} = 5$).	6
4	Árvore arterial com 10 terminais ($N_{term} = 10$).	7
5	Árvore arterial com 20 terminais ($N_{term} = 20$).	8
6	Árvore arterial com 50 terminais ($N_{term} = 50$).	9
7	Árvore arterial com 100 terminais ($N_{term} = 100$).	10

Lista de Códigos Fonte

1	Avaliação dos candidatos	12
2	Validação de interseção	13
3	Teste geométrico de interseção	13

1 Introdução

As árvores arteriais têm a função de transportar sangue aos tecidos, garantindo o suprimento de oxigênio e nutrientes necessários para o metabolismo celular. A complexidade da estrutura vascular observada na natureza levanta uma importante questão científica: por que os vasos sanguíneos apresentam determinados padrões de crescimento, ramificação e bifurcação? Estudos experimentais realizados em árvores arteriais humanas mostraram que as bifurcações seguem relações geométricas específicas envolvendo raios dos vasos, ângulos de ramificação e distribuição do fluxo sanguíneo. Essas características sugerem a existência de princípios de otimização capazes de explicar a organização da rede vascular.

O algoritmo CCO (Constrained Constructive Optimization) é um método utilizado para a geração de árvores arteriais artificiais. Seu objetivo é construir uma rede de vasos sanguíneos capaz de irrigar uma determinada região do domínio, respeitando restrições geométricas e fisiológicas. A ideia principal consiste em adicionar novos terminais à árvore de forma incremental. Para cada novo ponto gerado, o algoritmo busca a melhor conexão possível com a estrutura existente, criando bifurcações e novos segmentos. Durante esse processo, critérios de otimização são utilizados para obter uma árvore organizada e com características semelhantes às observadas em sistemas vasculares reais.

1.1 Considerações iniciais

Para a criação desse projeto, foram utilizadas algumas ferramentas de desenvolvimento, como:

- Ambiente de desenvolvimento do código fonte: **Visual Studio Code**.¹
- Linguagem utilizada: **C++**.
- Ambiente de desenvolvimento da documentação: **Overleaf L^AT_EX**.²

1.2 Instruções de compilação e execução

Para a compilação do projeto, foi utilizado um arquivo **Makefile**, com o objetivo de simplificar o processo de construção do executável. O arquivo contém as instruções necessárias para compilar todos os arquivos-fonte da aplicação e gerar o programa final.

Compilando o projeto

```
make
```

O comando acima executa as instruções definidas no arquivo *Makefile*, equivalente à compilação manual por meio do comando:

Compilação manual

```
g++ -I ../include *.cpp -o programa -lGL -lGLU -lglut
```

As diretivas utilizadas na compilação são:

- *g++*: compilador C++ utilizado no desenvolvimento do projeto;
- *-I ../include*: indica ao compilador o diretório onde se encontram os arquivos de cabeçalho (.hpp);
- **.cpp*: seleciona automaticamente todos os arquivos-fonte da pasta atual;
- *-o programa*: define o nome do executável gerado;
- *-lGL -lGLU -lglut*: realiza a ligação das bibliotecas OpenGL utilizadas pelo projeto.

¹Visual Studio Code está disponível em <https://code.visualstudio.com/>

²Disponível em <https://www.overleaf.com/>

Após a compilação, a execução da simulação é realizada informando a quantidade de terminais desejada e o raio do domínio circular:

Execução do programa

```
./programa <numero_de_terminais> <raio>
```

Por exemplo:

Exemplo de execução

```
./programa 10 10
```

Neste exemplo, será gerada uma árvore arterial contendo 10 terminais em um domínio circular de raio igual a 10 unidades.

Após a execução, são gerados os arquivos `arvore.vtk` e `raio.txt`, utilizados na etapa de visualização da árvore. A visualização gráfica é realizada através do script Python:

Visualização da árvore

```
python3 visualizar.py arvore.vtk
```

O comando abre uma janela gráfica contendo a árvore arterial gerada, os segmentos da estrutura vascular e o domínio circular utilizado durante a simulação.

2 Descrição da implementação

O modelo inicial simplificado inspirado no método CCO (Constrained Constructive Optimization), foi desenvolvido em C++ para a geração da árvore arterial e em Python para a visualização gráfica dos resultados. A estrutura do código foi organizada em duas pastas principais:

- **include/**: contém os arquivos de cabeçalho (.hpp), responsáveis pelas definições das estruturas de dados e protótipos das funções do sistema.
- **src/**: contém os arquivos de implementação (.cpp), o arquivo Makefile e o script de visualização.

2.1 Organização dos Arquivos

A pasta include contém os seguintes arquivos:

- **point.hpp**: definição da estrutura de pontos bidimensionais.
- **segment.hpp**: representação dos segmentos da árvore arterial.
- **node.hpp**: estrutura dos nós utilizados na árvore.
- **candidate.hpp**: definição dos candidatos avaliados para conexão de novos terminais.
- **tree.hpp**: estrutura principal da árvore arterial e protótipos das operações de manipulação.
- **geometry.hpp**: funções geométricas utilizadas nos cálculos de distância e interseção.
- **domain.hpp**: funções relacionadas ao domínio circular da simulação.
- **random.hpp**: funções auxiliares para geração de valores aleatórios.

A pasta src contém:

- **main.cpp**: implementação principal do algoritmo de crescimento da árvore.
- **tree.cpp**: operações de inserção de nós, geração de candidatos e exportação da árvore.

- **geometry.cpp:** implementação dos cálculos geométricos.
- **domain.cpp:** geração de pontos válidos dentro do domínio.
- **random.cpp:** funções auxiliares para sorteio de coordenadas.
- **visualizar.py:** responsável pela visualização gráfica da árvore gerada.
- **Makefile:** automatiza o processo de compilação.

Inicialmente é criado um nó de entrada, denominado raiz da árvore, posicionado em:

$$(0, R)$$

Esse nó representa a entrada principal da circulação e é destacado em laranja durante a visualização.

Geração dos Terminais

O crescimento da árvore ocorre de forma iterativa. A cada iteração é gerado um novo terminal candidato dentro do domínio circular.

Diferentemente de abordagens baseadas em posições pré-definidas, os pontos não são inseridos utilizando coordenadas fixas. Cada novo terminal é obtido através de uma busca aleatória, sendo aceito somente quando satisfaz um conjunto de restrições geométricas.

Entre os critérios utilizados destacam-se:

- Distância mínima em relação aos terminais já existentes;
- Distância mínima em relação aos segmentos da árvore;
- Distância mínima em relação ao ponto de entrada da árvore;
- Respeito aos limites do domínio circular.

Para a geração do primeiro terminal também são aplicadas restrições adicionais para evitar que ele seja criado excessivamente próximo da entrada ou do centro geométrico do domínio.

Seleção da Melhor Conexão

Após a geração de um terminal válido, o algoritmo avalia todos os segmentos existentes na árvore como possíveis candidatos para conexão.

Para cada segmento é calculado um custo baseado na distância entre o novo terminal e o segmento avaliado. Em seguida são descartadas as conexões que produziram interseções com segmentos já existentes.

Os candidatos válidos são armazenados em uma lista e o segmento com menor custo é selecionado através de uma estratégia gulosa (*greedy*).

Inserção de Bifurcações

Quando um segmento é escolhido para receber uma nova conexão, ele é dividido em seu ponto médio, gerando uma bifurcação.

O segmento original é removido e substituído por três novos segmentos:

- segmento entre o início original e o ponto de bifurcação;
- segmento entre o ponto de bifurcação e o final original;
- novo ramo conectando o ponto de bifurcação ao terminal recém-gerado.

Esse processo é repetido até que a quantidade de terminais especificada pelo usuário seja atingida.

Exportação e Visualização

Ao final da execução, a árvore construída é exportada para um arquivo no formato VTK (*Visualization Toolkit*), permitindo sua visualização através do software desenvolvido em Python utilizando a biblioteca **PyVista**.

Na visualização:

- a raiz é exibida em laranja;
- os demais nós são exibidos em verde;
- os segmentos recebem cores alternadas para facilitar a identificação da estrutura;
- o domínio circular é representado por uma circunferência de contorno preto.

Dessa forma, é possível analisar visualmente o crescimento e a organização da árvore arterial gerada pelo algoritmo.

3 Exemplos de execução

Para avaliar o comportamento do algoritmo implementado, foram realizados experimentos utilizando um domínio circular de raio $R = 10$ e diferentes quantidades de terminais. As figuras a seguir apresentam exemplos das árvores geradas.

Caso 1: ($N_{term} = 1$)

A Figura 1 apresenta o caso mais simples possível, contendo apenas um terminal.

Neste cenário existe apenas um segmento conectando o nó de entrada (raiz) ao terminal gerado aleatoriamente dentro do domínio. Como não existem outros terminais, ainda não ocorre nenhuma bifurcação. Este caso serve como base para o crescimento incremental da árvore.

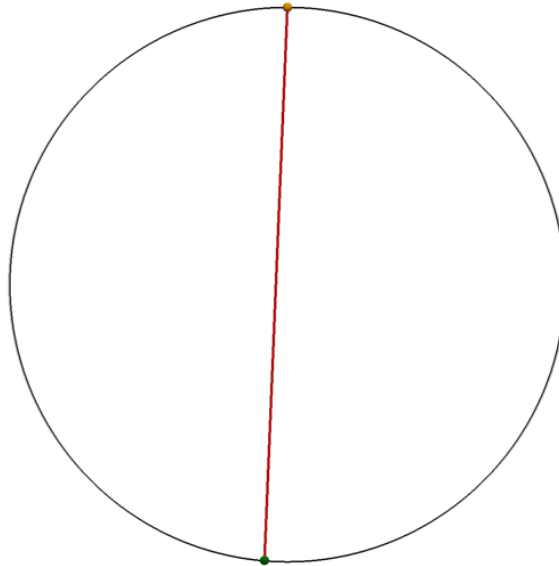


Figura 1: Árvore arterial com 1 terminal ($N_{term} = 1$).

Caso 2: ($N_{term} = 2$)

A Figura 2 mostra a árvore contendo dois terminais.

Após a geração do segundo terminal, o algoritmo avalia os segmentos existentes e seleciona aquele com menor custo de conexão. Como existe apenas um segmento disponível, ele é escolhido para receber a nova conexão.

O segmento original é então dividido em seu ponto médio, gerando a primeira bifurcação da árvore. O resultado é uma estrutura em formato de "Y", composta por três segmentos e dois terminais.

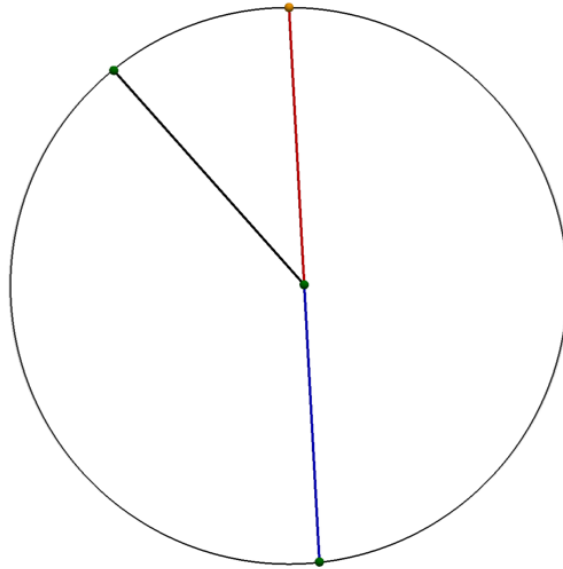


Figura 2: Árvore arterial com 2 terminais ($N_{term} = 2$).

Caso 3: ($N_{term} = 5$)

A Figura 3 apresenta a árvore contendo cinco terminais.

Neste estágio já é possível observar múltiplas bifurcações distribuídas pela estrutura. Para cada novo terminal inserido, o algoritmo avalia todos os segmentos candidatos, descarta aqueles que produziam interseções e seleciona o segmento de menor custo.

A árvore começa a ocupar diferentes regiões do domínio circular, mantendo espaçamento entre os terminais e evitando cruzamentos entre ramos.

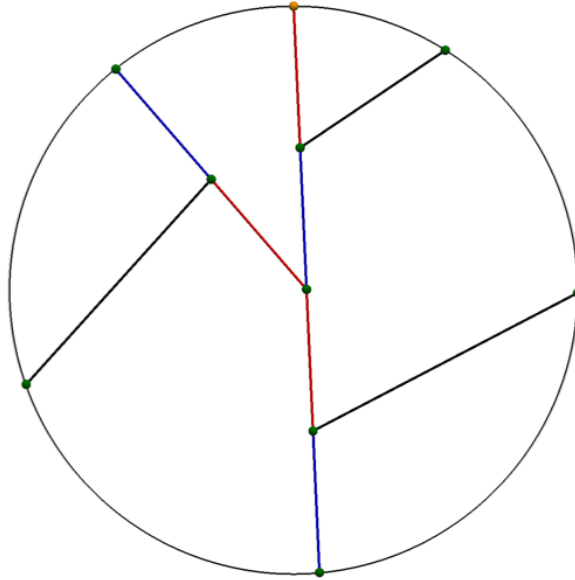


Figura 3: Árvore arterial com 5 terminais ($N_{term} = 5$).

Caso 4: ($N_{term} = 10$)

A Figura 4 apresenta a árvore com dez terminais.

Observa-se uma expansão mais significativa da estrutura vascular. O aumento do número de segmentos oferece mais alternativas de conexão, permitindo que novos terminais sejam inseridos em diferentes regiões do domínio.

As bifurcações tornam-se mais frequentes e a árvore passa a apresentar uma organização mais semelhante a uma rede arterial.

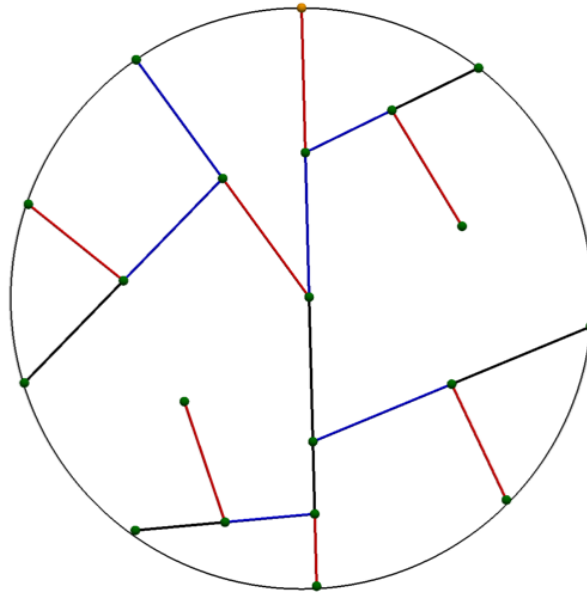


Figura 4: Árvore arterial com 10 terminais ($N_{term} = 10$).

Caso 5: ($N_{term} = 20$)

A Figura 5 mostra o resultado para vinte terminais.

Neste estágio a árvore já ocupa uma parcela significativa do domínio circular. Como a quantidade de terminais aumenta, o critério de distância mínima utilizado pelo algoritmo é gradualmente reduzido, permitindo a inserção de novos pontos em regiões mais próximas de estruturas já existentes.

Mesmo com essa redução, os terminais permanecem bem distribuídos e os segmentos continuam apresentando um espaçamento adequado, evitando concentrações excessivas em uma única região.

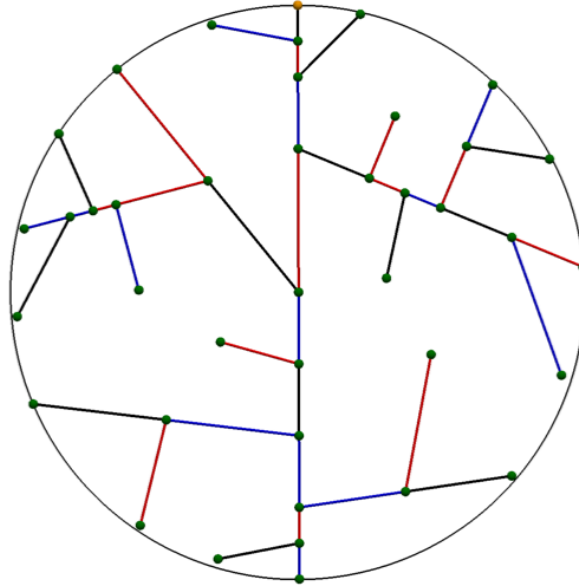


Figura 5: Árvore arterial com 20 terminais ($N_{term} = 20$).

Caso 6: ($N_{term} = 50$)

A Figura 6 apresenta a árvore gerada com cinquenta terminais.

Com o aumento do número de terminais, o critério de distância mínima torna-se mais permissivo, possibilitando a criação de segmentos menores e de novas bifurcações em áreas já parcialmente ocupadas.

Observa-se uma ocupação mais uniforme do domínio, com a árvore expandindo sua cobertura sem gerar cruzamentos entre os ramos. Os segmentos tornam-se progressivamente menores, porém continuam bem espaçados em relação aos demais elementos da estrutura.

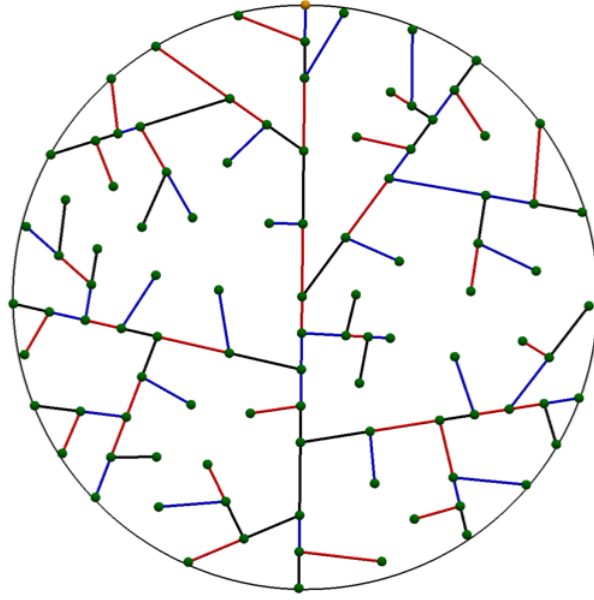


Figura 6: Árvore arterial com 50 terminais ($N_{term} = 50$).

Caso 7: ($N_{term} = 100$)

A Figura 7 apresenta a árvore contendo cem terminais.

Neste cenário, a redução gradual dos critérios de distância permite uma ocupação muito mais densa do domínio circular. Como consequência, surgem diversos segmentos curtos e uma quantidade elevada de bifurcações.

Percebe-se que a árvore passa a crescer não apenas em profundidade, mas também lateralmente, expandindo-se para diferentes regiões do domínio. Isso produz uma estrutura mais compacta e ramificada, capaz de preencher grande parte da área disponível mantendo a conectividade e evitando interseções entre segmentos.

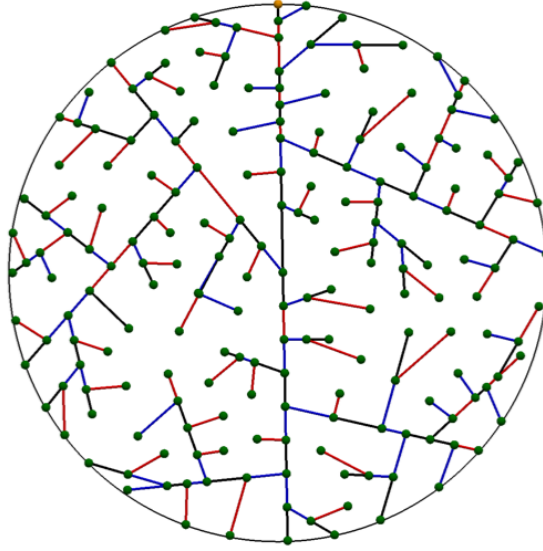


Figura 7: Árvore arterial com 100 terminais ($N_{term} = 100$).

4 Análise de Resultados

A análise de resultados foi realizada por meio de testes progressivos com diferentes quantidades de terminais. A seguir, detalham-se as saídas do programa para os cenários de 1, 5 e 10 terminais.

4.1 Análise do Relatório Final e a Equação de Nós

Um aspecto fundamental observado em todas as execuções é a relação entre o número de terminais solicitados e o número total de nós gerados na árvore. Notavelmente, ao solicitar N terminais, o programa reporta exatamente $2N$ nós totais (por exemplo, 5 terminais geram 10 nós, e 10 terminais geram 20 nós).

Essa propriedade é explicada pela topologia de uma árvore binária estrita gerada pelo modelo CCO. A estrutura é regida pela seguinte fórmula matemática:

$$Total_de_Nos = 2 \times N_{folhas}$$

A razão para isso ocorrer é que cada novo terminal aceito não apenas adiciona ele mesmo à árvore, mas também força a criação de um **nó interno de bifurcação** no segmento alvo onde ele se conectou. Como a raiz inicial é fixa, a adição de cada folha subsequente equilibra a topologia criando exatamente um par de nós (1 nó interno + 1 nó terminal), fazendo com que o total sempre dobre.

4.2 Cenário 1: Simulação com 1 Terminal

A execução inicial serve para validar o critério de afastamento da entrada, forçando o primeiro ponto a colonizar o lado oposto do círculo.

Saída do programa: 1 Terminal

```
[SUCESSO] Primeiro Terminal Gerado por Critério! -> Coordenadas: (-0.150643,
-9.98562) -> Tentativas gastas para achar: 5000
[INFO] Execucao finalizada com 1 terminal conforme solicitado. Numero total de nos: 2
Numero de folhas: 1 Arquivo VTK exportado com sucesso!
```

Componentes do relatório:

- *Primeiro Terminal Gerado*: O ponto nasce próximo a $(0, -10)$, que é o extremo oposto da raiz $(0, 10)$, validando o critério geométrico de distância máxima inicial.
- *Tentativas gastas*: Registra o esforço computacional no loop de sorteio estocástico.
- *Número total de nós (2)*: Representa o nó da Entrada (raiz) e o único nó Terminal (folha) conectado a ela.

4.3 Cenário 2: Simulação com 5 Terminais

Neste teste, observa-se o início das ramificações e a primeira ocorrência de rejeição de segmentos por cruzamento (interseção geométrica).

Saída do programa: 5 Terminais

```
[SUCESSO] Primeiro Terminal Gerado por Critério
Coordenadas: (-0.401882, -9.98803)
Candidatos para o ponto: (9.49622, -2.90736)
[REJEITADO] Raiz->A.1 -> cruza outro segmento
[1] Raiz->A.2 | custo = 9.75376
[2] Ramo->B | custo = 9.63373
Total de candidatos válidos: 2
Melhor segmento escolhido: Ramo->B
Número total de nós: 10
Número de folhas: 5
Comprimento total: 57.9686
Conexões rejeitadas: 4
Arquivo VTK exportado com sucesso.
```

Componentes do relatório:

- *Candidatos para o ponto*: lista os segmentos existentes aos quais o novo terminal poderia ser conectado.
- *[REJEITADO] ... cruza outro segmento*: indica que a conexão foi descartada pelo teste de interseção geométrica.
- *Comprimento total*: soma dos comprimentos de todos os segmentos presentes na árvore.
- *Conexões rejeitadas*: quantidade de candidatos descartados por violarem as restrições geométricas da estrutura.

4.4 Cenário 3: Simulação com 10 Terminais

Com 10 terminais, a árvore passa a ocupar o domínio de forma mais densa, aumentando a quantidade de segmentos candidatos avaliados a cada inserção.

Saída do programa: 10 Terminais

```
[SUCESSO] Iteração 10 -> Novo Ponto Aceito: Ponto J
Coordenadas finais: (3.74814, -3.87854)
Candidatos para o ponto: (3.74814, -3.87854)
[1] Ramo->B_1 | custo = 5.69491
[REJEITADO] Ramo->B_2 -> cruza outro segmento
[2] Raiz->A_2.1 | custo = 4.30347
[4] Ramo->G | custo = 4.07904
Total de candidatos válidos: 6
Melhor segmento escolhido: Ramo->G
Distância de conexão: 4.07904
Árvore atualizada. Total de folhas: 10
Número total de nós: 20
Número de folhas: 10
Comprimento total: 80.3275
Conexões rejeitadas: 18
Arquivo VTK exportado com sucesso.
```

Componentes do relatório:

- *Total de candidatos válidos*: mostra quantos segmentos puderam ser considerados para receber a nova conexão.
- *Melhor segmento escolhido*: representa o segmento selecionado pelo algoritmo por apresentar o menor custo de conexão.
- *Distância de conexão*: valor utilizado para comparar os candidatos e escolher a ligação mais adequada.
- *Conexões rejeitadas*: aumenta conforme a árvore cresce, pois há mais segmentos ocupando o domínio e mais possibilidades de cruzamentos.

5 Discussão sobre Complexidade

Nesta seção são analisados os dois principais componentes do algoritmo: a busca de candidatos para inserção de novos terminais e o teste de interseção entre segmentos.

5.1 Busca de Candidatos

A busca de candidatos ocorre durante a geração de cada novo terminal. O algoritmo realiza diversas tentativas aleatórias dentro do domínio circular e seleciona o ponto que apresenta o melhor espalhamento em relação à árvore já construída.

O trecho responsável por essa avaliação é apresentado a seguir.

```
1 if(dTerminal >= limiteTerminal &&
2   dSegmento >= 0.01 * R){
3
4     double score =
5       dTerminal + 0.35 * dSegmento;
6
7     if(score > melhorScore){
8       melhorScore = score;
9       melhorPonto = novoPonto;
10    }
11 }
```

Código 1: Avaliação dos candidatos

Para cada tentativa são calculadas:

- a menor distância até os terminais existentes;
- a menor distância até os segmentos existentes.

Como essas verificações percorrem toda a árvore, o custo de cada tentativa cresce proporcionalmente ao número de segmentos já existentes.

Complexidade da Busca de Candidatos

Complexidade por tentativa: $\mathcal{O}(N)$
Complexidade por inserção: $\mathcal{O}(M \cdot N)$
Complexidade total da árvore: $\mathcal{O}(N^2)$

onde:

- N representa a quantidade de segmentos da árvore;
- M representa o número máximo de tentativas realizadas para encontrar um novo terminal.

Como o valor de M é fixado no programa (`maxTentativas = 9000`), ele pode ser tratado como constante. Dessa forma, o crescimento da complexidade é aproximadamente quadrático em relação ao número de terminais gerados.

5.2 Teste de Interseção

Após a seleção de um novo terminal, o algoritmo gera uma lista de segmentos candidatos para realizar a conexão. Antes de aceitar qualquer ligação, é necessário verificar se o novo ramo não cruza segmentos já existentes.

O procedimento é realizado pelo trecho a seguir.

```
1 for(Segment& s : arvore.segmentos){
2
3     ...
4
5     for(Segment& existente : arvore.segmentos){
6
7         if(&existente == &s)
8             continue;
9
10        if(intersecaoSegmentos(
11            novoRamo, existente)){
12
13            intersecta = true;
14            break;
15        }
16    }
17 }
```

Código 2: Validação de interseção

A função responsável por verificar se dois segmentos se cruzam utiliza apenas operações geométricas simples baseadas na orientação de pontos.

```
1 return (o1 * o2 < 0 &&
2        o3 * o4 < 0);
```

Código 3: Teste geométrico de interseção

Como esse cálculo utiliza apenas multiplicações e comparações, seu custo é constante.

Complexidade do Teste de Interseção

Teste entre dois segmentos: $\mathcal{O}(1)$
Validação de todos os candidatos: $\mathcal{O}(N^2)$
Complexidade acumulada da construção: $\mathcal{O}(N^3)$

O comportamento quadrático ocorre porque, para cada segmento candidato, o algoritmo compara o novo ramo com todos os demais segmentos já existentes na árvore.

Na prática, para árvores pequenas e médias (até algumas centenas de terminais), o desempenho permanece satisfatório. Entretanto, à medida que o número de ramificações cresce, o teste de interseção passa a representar a etapa mais custosa da simulação.

6 Considerações Finais

Neste trabalho foi desenvolvida uma implementação simplificada do algoritmo CCO (*Constrained Constructive Optimization*) para a geração de árvores arteriais em um domínio circular. A abordagem adotada permitiu construir estruturas ramificadas de forma incremental, conectando novos terminais à árvore existente por meio de critérios geométricos de distância e validação de interseções. Os resultados obtidos demonstraram que o algoritmo é capaz de gerar árvores conectadas e bem distribuídas dentro do domínio, preservando características típicas de sistemas arteriais, como a presença de bifurcações e a expansão progressiva da estrutura à medida que novos terminais são inseridos. Observou-se também que o aumento do número de terminais provoca um crescimento significativo da quantidade de candidatos avaliados e das conexões rejeitadas, principalmente devido ao critério de não interseção entre segmentos. Além disso, a redução gradual das distâncias mínimas permitiu a inserção de novos terminais em regiões cada vez mais densas do domínio, produzindo árvores mais compactas e ramificadas.